

作业感受

(1) 简述你是如何完成第一次挑战的

基本上是摸着石头过来的。机器上没有 GPU，只有 4 核和 8 GB 的容器限额，而 SGLang 的文档默认你在用 CUDA。我先照文档装依赖、拉 Qwen3-0.6B，然后问题一个接一个：缺 `pybase64`、缺 `compressed_tensors`，最莫名其妙的是报 `No module named vllm`——我完全没往 CPU 后端上想，是顺着报错栈摸到 `srt/utils` 里的 `is_cpu()`，才发现 CPU 引擎要靠 `SGLANG_USE_CPU_ENGINE=1` 手动打开。服务好不容易起来了又会被杀，`free` 显示 100 多 GB 可用，怎么看都不像内存问题，最后是读 `cgroup` 的 `memory.events` 才看到 `oom_kill` 计数在涨，把 KV 池压到 16384 token 才稳住。

跑通 `/v1/models` 和一次对话之后，我写了个回放脚本：从 Mooncake trace 里挑 24 条真实记录，按 `input_length` 拼出对应长度的 prompt，用指数分布的间隔模拟泊松到达，逐条记 TTFT 和 latency。第一版我把平均间隔设成 3 秒，结果系统直接过载，TTFT 涨到 9.7 秒、p95 快到 150 秒——那已经不是服务性能，是排队时间。改成 18 秒之后指标才正常。最后再对照源码画流程图、整理自己对 RadixAttention 的理解。整个过程中 AI 主要帮我干两件事：把报错翻译成“该去哪看代码”，以及在几万行源码里指出该读哪几个函数。

(2) 哪部分最困难，如何克服

最卡的是让服务稳定跑起来那一段。难点不在报错有多奇怪，而在报错信息和真实原因对不上：进程 `exit -9`，`free` 却说内存很充裕。我一开始以为是 torch 或 Ray 的问题，来回重装了几次都没用。后来换个思路，先不管 SGLang，直接去确认“这台机器到底给我多少资源”——读 `cgroup` 的 `memory.max`，8 GB；再读 `memory.events`，`oom_kill` 已经是 2 了。到这一步才算对症：把 `mem-fraction-static` 压到 0.045、`max-total-tokens` 设成 16384，服务就再没被杀过。另一个坑也很费时间：明明代码里有 `is_cpu()` 分支却始终走不到，因为那个分支要求显式设环境变量。

这两件事给我的教训其实是一句话——报错只是入口，真正要问的是“这个系统以为自己在什么环境里、有多少资源”，而不是“我有没有按文档做”。

(3) 从 0 到 1 的科研工作需要哪些方面，对今后科研的启发

这次做完，我觉得一件从 0 到 1 的事至少得有三样东西撑着。一是可复现：版本、启动参数、数据来源、随机种子都得写下来，否则别人（包括两周后的我自己）复现不出来，结论就站不住，我把启动脚本和日志都留在仓库里就是这个原因。二是可测量：先想清楚用什么指标说话，再动手，而且最好只改一个变量；HW2 的两组负载就是这个思路，除“前缀能不能复用”之外其余全一样，差异才敢归因到前缀缓存上。三是能解释：光说“快了 64.9%”没意义，得能从因果自注意力的公式一路推到 `schedule_batch.py` 里那行 `set_extend_input_len`，才算真的懂了。

对我自己做研究的启发有两点。一个是别急着优化，先把系统跑通、把测量口径定好再谈结论——我第一版 workload 过载那次，差点就把排队时间当成服务延迟写进报告。另一个是要留住那些不好看的结果：过载的那组数据、被误判的 `flush_cache`、还有共享组命中 919.5 token（比设计的 896 还多）这个反直觉现象，最后都成了报告里最有意思的部分，因为它们逼着我去搞清楚 radix tree 到底缓存了什么。